import python library

In [2]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

In [5]:
```
import os
print(os.getcwd())
```

```
C:\Users\Balaji\Downloads\QVI_Chip_Project
```

In [6]:
```
import os

print(os.listdir())
```

```
['.ipynb_checkpoints', 'QVI_analysis.ipynb', 'QVI_purchase_behaviour.csv', 'QVI_transaction_data.xlsx']
```

Step 2: Load the Datasets

In [9]:
```
transaction_data = pd.read_excel("QVI_transaction_data.xlsx")

customer_data = pd.read_csv("QVI_purchase_behaviour.csv")
```

Step 3: Data Quality Checks

"""The following checks were performed:

- Dataset dimensions
- Column names
- Data types
- Missing values
- Duplicate records """

In [11]:
```
transaction_data.head()
```

Out[11]:

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	PROD_NAME	PROD_C
0	43390	1	1000	1	5	Natural Chip Compny SeaSalt175g	
1	43599	1	1307	348	66	CCs Nacho Cheese 175g	
2	43605	1	1343	383	61	Smiths Crinkle Cut Chips Chicken 170g	
3	43329	2	2373	974	69	Smiths Chip Thinly S/Cream&Onion 175g	
4	43330	2	2426	1038	108	Kettle Tortilla ChpsHny&Jlpno Chili 150g	

In [12]:
```
#customer data
```

In [13]:
```
customer_data.head()
```

Out[13]:

	LYLTY_CARD_NBR	LIFESTAGE	PREMIUM_CUSTOMER
0	1000	YOUNG SINGLES/COUPLES	Premium
1	1002	YOUNG SINGLES/COUPLES	Mainstream
2	1003	YOUNG FAMILIES	Budget
3	1004	OLDER SINGLES/COUPLES	Mainstream
4	1005	MIDAGE SINGLES/COUPLES	Mainstream

In [14]:
```
# data set size or shape
```

In [16]:
```
transaction_data.shape
```

Out[16]:
```
(264836, 8)
```

In [17]:
```
customer_data.shape
```

Out[17]:
```
(72637, 3)
```

In [18]:
```
# columes name in data set
```

In [20]:
```
transaction_data.columns
```

Out[20]:
```
Index(['DATE', 'STORE_NBR', 'LYLTY_CARD_NBR', 'TXN_ID', 'PROD_NBR',
       'PROD_NAME', 'PROD_QTY', 'TOT_SALES'],
      dtype='object')
```

In [21]:
```
customer_data.columns
```

Out[21]:
```
Index(['LYLTY_CARD_NBR', 'LIFESTAGE', 'PREMIUM_CUSTOMER'], dtype='object')
```

In [22]:
```
# First 10 rows of datasetes
```

In [23]:
```
transaction_data.head(10)
```

Out[23]:

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	PROD_NAME	PROD_C
0	43390	1	1000	1	5	Natural Chip Compny SeaSalt175g	
1	43599	1	1307	348	66	CCs Nacho Cheese 175g	
2	43605	1	1343	383	61	Smiths Crinkle Cut Chips Chicken 170g	
3	43329	2	2373	974	69	Smiths Chip Thinly S/Cream&Onion 175g	
4	43330	2	2426	1038	108	Kettle Tortilla ChpsHny&Jlpno Chili 150g	
5	43604	4	4074	2982	57	Old El Paso Salsa Dip Tomato Mild 300g	
6	43601	4	4149	3333	16	Smiths Crinkle Chips Salt & Vinegar 330g	
7	43601	4	4196	3539	24	Grain Waves Sweet Chilli 210g	
8	43332	5	5026	4525	42	Doritos Corn Chip Mexican Jalapeno 150g	
9	43330	7	7150	6900	52	Grain Waves Sour Cream&Chives 210G	

In [24]:
```
customer_data.head(10)
```

Out[24]:

	LYLTY_CARD_NBR	LIFESTAGE	PREMIUM_CUSTOMER
0	1000	YOUNG SINGLES/COUPLES	Premium
1	1002	YOUNG SINGLES/COUPLES	Mainstream
2	1003	YOUNG FAMILIES	Budget
3	1004	OLDER SINGLES/COUPLES	Mainstream
4	1005	MIDAGE SINGLES/COUPLES	Mainstream
5	1007	YOUNG SINGLES/COUPLES	Budget
6	1009	NEW FAMILIES	Premium
7	1010	YOUNG SINGLES/COUPLES	Mainstream
8	1011	OLDER SINGLES/COUPLES	Mainstream
9	1012	OLDER FAMILIES	Mainstream

In [25]:
```
# Last 10 rows
```

In [26]:
```
transaction_data.tail(10)
```

Out[26]:

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	PROD_NAME
264826	43549	272	272194	269908	75	Cobs Popd Sea Salt Chips 110g
264827	43340	272	272197	269911	104	Infuzions Thai SweetChili PotatoMix 110g
264828	43308	272	272236	269974	68	Pringles Chicken Salt Crips 134g
264829	43540	272	272236	269976	49	Infuzions SourCream&Herbs Veg Strws 110g
264830	43416	272	272319	270087	44	Thins Chips Light& Tangy 175g
264831	43533	272	272319	270088	89	Kettle Sweet Chilli And Sour Cream 175g
264832	43325	272	272358	270154	74	Tostitos Splash Of Lime 175g
264833	43410	272	272379	270187	51	Doritos Mexicana 170g
264834	43461	272	272379	270188	42	Doritos Corn Chip Mexican Jalapeno 150g
264835	43365	272	272380	270189	74	Tostitos Splash Of Lime 175g

In [27]:
```
customer_data.tail(10)
```

Out[27]:

	LYLTY_CARD_NBR	LIFESTAGE	PREMIUM_CUSTOMER
72627	2330501	OLDER SINGLES/COUPLES	Budget
72628	2370001	OLDER SINGLES/COUPLES	Premium
72629	2370181	YOUNG SINGLES/COUPLES	Mainstream
72630	2370361	OLDER SINGLES/COUPLES	Budget
72631	2370581	OLDER SINGLES/COUPLES	Budget
72632	2370651	MIDAGE SINGLES/COUPLES	Mainstream
72633	2370701	YOUNG FAMILIES	Mainstream
72634	2370751	YOUNG FAMILIES	Premium
72635	2370961	OLDER FAMILIES	Budget
72636	2373711	YOUNG SINGLES/COUPLES	Mainstream

In [28]:
```
# info check means data type
```

In [30]:
```
transaction_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 264836 entries, 0 to 264835
Data columns (total 8 columns):
 #   Column          Non-Null Count   Dtype  
---  ------          --------------   -----  
 0   DATE            264836 non-null  int64  
 1   STORE_NBR       264836 non-null  int64  
 2   LYLTY_CARD_NBR  264836 non-null  int64  
 3   TXN_ID          264836 non-null  int64  
 4   PROD_NBR        264836 non-null  int64  
 5   PROD_NAME       264836 non-null  object 
 6   PROD_QTY        264836 non-null  int64  
 7   TOT_SALES       264836 non-null  float64
dtypes: float64(1), int64(6), object(1)
memory usage: 16.2+ MB
```

In [31]:
```
customer_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 72637 entries, 0 to 72636
Data columns (total 3 columns):
 #   Column            Non-Null Count  Dtype 
---  ------            --------------  ----- 
 0   LYLTY_CARD_NBR    72637 non-null  int64 
 1   LIFESTAGE         72637 non-null  object
 2   PREMIUM_CUSTOMER  72637 non-null  object
dtypes: int64(1), object(2)
memory usage: 1.7+ MB
```

In [32]:
```
# checking null value
```

In [33]:
```
transaction_data.isnull().sum()
```

Out[33]:
```
DATE              0
STORE_NBR         0
LYLTY_CARD_NBR    0
TXN_ID            0
PROD_NBR          0
PROD_NAME         0
PROD_QTY          0
TOT_SALES         0
dtype: int64
```

In [35]:
```
customer_data.isnull().sum()
```

Out[35]:
```
LYLTY_CARD_NBR      0
LIFESTAGE           0
PREMIUM_CUSTOMER    0
dtype: int64
```

In [37]:
```
#
```

In [36]:
```
transaction_data['DATE'].head()
```

Out[36]:
```
0    43390
1    43599
2    43605
3    43329
4    43330
Name: DATE, dtype: int64
```

In [38]:
```
#duplicated value check
```

In [39]:
```
transaction_data.duplicated().sum()
```

Out[39]:
```
np.int64(1)
```

In [40]:
```
customer_data.duplicated().sum()
```

Out[40]:
```
np.int64(0)
```

In [41]:
```
#find duplicate value
```

In [42]:
```
transaction_data[transaction_data.duplicated()]
```

Out[42]:

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	PROD_NAME	PRO
124845	43374	107	107024	108462	45	Smiths Thinly Cut Roast Chicken 175g	

In [43]:
```
#remove duplicate data
```

In [44]:
```
transaction_data = transaction_data.drop_duplicates()
```

In [45]:
```
#check duplicate data
```

In [46]:
```
transaction_data.duplicated().sum()
```

Out[46]:
```
np.int64(0)
```

In [50]:
```
#data set details
```

In [48]:
```
transaction_data['PROD_QTY'].describe()
```

Out[48]:
```
count    264835.000000
mean          1.907308
std           0.643655
min           1.000000
25%           2.000000
50%           2.000000
75%           2.000000
max         200.000000
Name: PROD_QTY, dtype: float64
```

In [49]:
```
transaction_data['TOT_SALES'].describe()
```

Out[49]:
```
count    264835.000000
mean          7.304205
std           3.083231
min           1.500000
25%           5.400000
50%           7.400000
75%           9.200000
max         650.000000
Name: TOT_SALES, dtype: float64
```

Step 4: Remove Outliers

In [53]:
```
transaction_data[transaction_data['PROD_QTY'] == 200]
```

Out[53]:

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	PROD_NAME	PROD
69762	43331	226	226000	226201	4	Dorito Corn Chp Supreme 380g	
69763	43605	226	226000	226210	4	Dorito Corn Chp Supreme 380g	

In [ ]:
```
```

In [55]:
```
transaction_data[transaction_data['LYLTY_CARD_NBR'] == 226000]
```

Out[55]:

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	PROD_NAME	PROD
69762	43331	226	226000	226201	4	Dorito Corn Chp Supreme 380g	
69763	43605	226	226000	226210	4	Dorito Corn Chp Supreme 380g	

In [56]:
```
#check more than 50 pocket chip transaction
```

In [57]:
```
transaction_data[transaction_data['PROD_QTY'] > 50]
```

Out[57]:

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	PROD_NAME	PROD
69762	43331	226	226000	226201	4	Dorito Corn Chp Supreme 380g	
69763	43605	226	226000	226210	4	Dorito Corn Chp Supreme 380g	

In [58]:
```
#"Two transactions with unusually high purchase quantities (200 packs) were iden
```

In [59]:
```
transaction_data = transaction_data[transaction_data['PROD_QTY'] < 50]
```

In [60]:
```
transaction_data['PROD_QTY'].max()
```

Out[60]: 5

In [61]:
```
#data transformation
```

Step 5: Feature Engineering

"""Two new features were created:

- PACK_SIZE
- BRAND """

In [62]:
```
transaction_data['PROD_NAME'].head(10)
```

Out[62]:
```
0      Natural Chip        Compny SeaSalt175g
1                    CCs Nacho Cheese    175g
2      Smiths Crinkle Cut  Chips Chicken 170g
3      Smiths Chip Thinly  S/Cream&Onion 175g
4    Kettle Tortilla ChpsHny&Jlpno Chili 150g
5    Old El Paso Salsa   Dip Tomato Mild 300g
6    Smiths Crinkle Chips Salt & Vinegar 330g
7       Grain Waves         Sweet Chilli 210g
8     Doritos Corn Chip Mexican Jalapeno 150g
9       Grain Waves Sour    Cream&Chives 210G
Name: PROD_NAME, dtype: object
```

In [63]:
```
transaction_data['PROD_NAME'].nunique()
```

Out[63]: 114

In [64]:
```
#adding new column
```

In [65]:
```
transaction_data['PACK_SIZE'] = transaction_data['PROD_NAME'].str.extract(r'(\d+
```

In [70]:
```
transaction_data['PROD_NAME'] = transaction_data['PROD_NAME'].str.upper()
#upper case product name
```

In [68]:
```
transaction_data['PACK_SIZE'] = transaction_data['PROD_NAME'].str.extract(r'(\d+
#
```

In [72]:
```
transaction_data = pd.read_excel("QVI_transaction_data.xlsx")
```

In [73]:
```
transaction_data['PROD_NAME'].head()
```

Out[73]:
```
0      Natural Chip        Compny SeaSalt175g
1                    CCs Nacho Cheese    175g
2      Smiths Crinkle Cut  Chips Chicken 170g
3      Smiths Chip Thinly  S/Cream&Onion 175g
4    Kettle Tortilla ChpsHny&Jlpno Chili 150g
Name: PROD_NAME, dtype: object
```

In [74]:
```
transaction_data['PROD_NAME'] = transaction_data['PROD_NAME'].str.upper()
```

In [75]:
```
transaction_data['PACK_SIZE'] = transaction_data['PROD_NAME'].str.extract(r'(\d+
```

In [76]:
```
transaction_data[['PROD_NAME','PACK_SIZE']].head(10)
```

Out[76]:

	PROD_NAME	PACK_SIZE
0	NATURAL CHIP COMPNY SEASALT175G	175G
1	CCS NACHO CHEESE 175G	175G
2	SMITHS CRINKLE CUT CHIPS CHICKEN 170G	170G
3	SMITHS CHIP THINLY S/CREAM&ONION 175G	175G
4	KETTLE TORTILLA CHPSHNY&JLPNO CHILI 150G	150G
5	OLD EL PASO SALSA DIP TOMATO MILD 300G	300G
6	SMITHS CRINKLE CHIPS SALT & VINEGAR 330G	330G
7	GRAIN WAVES SWEET CHILLI 210G	210G
8	DORITOS CORN CHIP MEXICAN JALAPENO 150G	150G
9	GRAIN WAVES SOUR CREAM&CHIVES 210G	210G

In [79]:
```
transaction_data['PROD_NAME'].head(20)
```

Out[79]:
```
0       NATURAL CHIP        COMPNY SEASALT175G
1                     CCS NACHO CHEESE    175G
2       SMITHS CRINKLE CUT  CHIPS CHICKEN 170G
3       SMITHS CHIP THINLY  S/CREAM&ONION 175G
4     KETTLE TORTILLA CHPSHNY&JLPNO CHILI 150G
5     OLD EL PASO SALSA   DIP TOMATO MILD 300G
6     SMITHS CRINKLE CHIPS SALT & VINEGAR 330G
7        GRAIN WAVES         SWEET CHILLI 210G
8      DORITOS CORN CHIP MEXICAN JALAPENO 150G
9        GRAIN WAVES SOUR    CREAM&CHIVES 210G
10    SMITHS CRINKLE CHIPS SALT & VINEGAR 330G
11       KETTLE SENSATIONS   SIRACHA LIME 150G
12                    TWISTIES CHEESE     270G
13           WW CRINKLE CUT      CHICKEN 175G
14             THINS CHIPS LIGHT&  TANGY 175G
15                           CCS ORIGINAL 175G
16                           BURGER RINGS 220G
17      NCC SOUR CREAM &    GARDEN CHIVES 175G
18     DORITOS CORN CHIP SOUTHERN CHICKEN 150G
19                    CHEEZELS CHEESE BOX 125G
Name: PROD_NAME, dtype: object
```

In [80]:
```
# we see thst in product name there is pack size we have to remove it
```

In [81]:
```
transaction_data['PRODUCT_NO_SIZE'] = transaction_data['PROD_NAME'].str.replace(
```

In [82]:
```
#check
transaction_data[['PROD_NAME','PRODUCT_NO_SIZE']].head(10)
```

Out[82]:

	PROD_NAME	PRODUCT_NO_SIZE
0	NATURAL CHIP COMPNY SEASALT175G	NATURAL CHIP COMPNY SEASALT
1	CCS NACHO CHEESE 175G	CCS NACHO CHEESE
2	SMITHS CRINKLE CUT CHIPS CHICKEN 170G	SMITHS CRINKLE CUT CHIPS CHICKEN
3	SMITHS CHIP THINLY S/CREAM&ONION 175G	SMITHS CHIP THINLY S/CREAM&ONION
4	KETTLE TORTILLA CHPSHNY&JLPNO CHILI 150G	KETTLE TORTILLA CHPSHNY&JLPNO CHILI
5	OLD EL PASO SALSA DIP TOMATO MILD 300G	OLD EL PASO SALSA DIP TOMATO MILD
6	SMITHS CRINKLE CHIPS SALT & VINEGAR 330G	SMITHS CRINKLE CHIPS SALT & VINEGAR
7	GRAIN WAVES SWEET CHILLI 210G	GRAIN WAVES SWEET CHILLI
8	DORITOS CORN CHIP MEXICAN JALAPENO 150G	DORITOS CORN CHIP MEXICAN JALAPENO
9	GRAIN WAVES SOUR CREAM&CHIVES 210G	GRAIN WAVES SOUR CREAM&CHIVES

In [83]:
```
transaction_data['PRODUCT_NO_SIZE'].str.split().str[0].value_counts()
```

Out[83]:
```
PRODUCT_NO_SIZE
KETTLE        41288
SMITHS        28860
PRINGLES      25102
DORITOS       24962
THINS         14075
RRD           11894
INFUZIONS     11057
WW            10320
COBS           9693
TOSTITOS       9471
TWISTIES       9454
OLD            9324
TYRRELLS       6442
GRAIN          6272
NATURAL        6050
RED            5885
CHEEZELS       4603
CCS            4551
WOOLWORTHS     4437
DORITO         3185
INFZNS         3144
SMITH          2963
CHEETOS        2927
SNBTS          1576
BURGER         1564
GRNWVES        1468
SUNBITES       1432
NCC            1419
FRENCH         1418
Name: count, dtype: int64
```

In [84]:
```
transaction_data['PRODUCT_NO_SIZE'].str.split().str[0].value_counts().head(20)
```

Out[84]:
```
PRODUCT_NO_SIZE
KETTLE        41288
SMITHS        28860
PRINGLES      25102
DORITOS       24962
THINS         14075
RRD           11894
INFUZIONS     11057
WW            10320
COBS           9693
TOSTITOS       9471
TWISTIES       9454
OLD            9324
TYRRELLS       6442
GRAIN          6272
NATURAL        6050
RED            5885
CHEEZELS       4603
CCS            4551
WOOLWORTHS     4437
DORITO         3185
Name: count, dtype: int64
```

In [85]:
```
transaction_data['BRAND'] = transaction_data['PRODUCT_NO_SIZE'].str.split().str[
```

In [86]:
```
transaction_data[['PRODUCT_NO_SIZE','BRAND']].head(20)
```

Out[86]:

	PRODUCT_NO_SIZE	BRAND
0	NATURAL CHIP COMPNY SEASALT	NATURAL
1	CCS NACHO CHEESE	CCS
2	SMITHS CRINKLE CUT CHIPS CHICKEN	SMITHS
3	SMITHS CHIP THINLY S/CREAM&ONION	SMITHS
4	KETTLE TORTILLA CHPSHNY&JLPNO CHILI	KETTLE
5	OLD EL PASO SALSA DIP TOMATO MILD	OLD
6	SMITHS CRINKLE CHIPS SALT & VINEGAR	SMITHS
7	GRAIN WAVES SWEET CHILLI	GRAIN
8	DORITOS CORN CHIP MEXICAN JALAPENO	DORITOS
9	GRAIN WAVES SOUR CREAM&CHIVES	GRAIN
10	SMITHS CRINKLE CHIPS SALT & VINEGAR	SMITHS
11	KETTLE SENSATIONS SIRACHA LIME	KETTLE
12	TWISTIES CHEESE	TWISTIES
13	WW CRINKLE CUT CHICKEN	WW
14	THINS CHIPS LIGHT& TANGY	THINS
15	CCS ORIGINAL	CCS
16	BURGER RINGS	BURGER
17	NCC SOUR CREAM & GARDEN CHIVES	NCC
18	DORITOS CORN CHIP SOUTHERN CHICKEN	DORITOS
19	CHEEZELS CHEESE BOX	CHEEZELS

In [87]:
```
brand_replace = {
    'OLD': 'OLD EL PASO',
    'NATURAL': 'NATURAL CHIP COMPANY',
    'GRAIN': 'GRAIN WAVES',
    'BURGER': 'BURGER RINGS',
    'WW': 'WOOLWORTHS'
}

transaction_data['BRAND'] = transaction_data['BRAND'].replace(brand_replace)
```

In [88]:
```
transaction_data['BRAND'].value_counts().head(20)
```

Out[88]:
```
BRAND
KETTLE                  41288
SMITHS                  28860
PRINGLES                25102
DORITOS                 24962
WOOLWORTHS              14757
THINS                   14075
RRD                     11894
INFUZIONS               11057
COBS                     9693
TOSTITOS                 9471
TWISTIES                 9454
OLD EL PASO              9324
TYRRELLS                 6442
GRAIN WAVES              6272
NATURAL CHIP COMPANY     6050
RED                      5885
CHEEZELS                 4603
CCS                      4551
DORITO                   3185
INFZNS                   3144
Name: count, dtype: int64
```

In [89]:
```
brand_fix = {
    'DORITO': 'DORITOS',
    'INFZNS': 'INFUZIONS'
}

transaction_data['BRAND'] = transaction_data['BRAND'].replace(brand_fix)
```

In [90]:
```
transaction_data['BRAND'].value_counts().head(20)
```

Out[90]:
```
BRAND
KETTLE                  41288
SMITHS                  28860
DORITOS                 28147
PRINGLES                25102
WOOLWORTHS              14757
INFUZIONS               14201
THINS                   14075
RRD                     11894
COBS                     9693
TOSTITOS                 9471
TWISTIES                 9454
OLD EL PASO              9324
TYRRELLS                 6442
GRAIN WAVES              6272
NATURAL CHIP COMPANY     6050
RED                      5885
CHEEZELS                 4603
CCS                      4551
SMITH                    2963
CHEETOS                  2927
Name: count, dtype: int64
```

In [91]:
```
transaction_data['BRAND'] = transaction_data['BRAND'].replace({
    'SMITH': 'SMITHS'
})
```

In [92]:
```
transaction_data['BRAND'].value_counts().head(20)
```

Out[92]:
```
BRAND
KETTLE                41288
SMITHS                31823
DORITOS               28147
PRINGLES              25102
WOOLWORTHS            14757
INFUZIONS             14201
THINS                 14075
RRD                   11894
COBS                   9693
TOSTITOS               9471
TWISTIES               9454
OLD EL PASO            9324
TYRRELLS               6442
GRAIN WAVES            6272
NATURAL CHIP COMPANY   6050
RED                    5885
CHEEZELS               4603
CCS                    4551
CHEETOS                2927
SNBTS                  1576
Name: count, dtype: int64
```

Step 6: Merge Transaction and Customer Data

In [96]:
```
data = transaction_data.merge(
    customer_data,
    on='LYLTY_CARD_NBR',
    how='left'
)
# merge the data
```

In [97]:
```
data.shape
```

Out[97]:
```
(264836, 13)
```

In [98]:
```
data.head()
```

Out[98]:

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	PROD_NAME	PROD
0	43390	1	1000	1	5	NATURAL CHIP COMPNY SEASALT175G	
1	43599	1	1307	348	66	CCS NACHO CHEESE 175G	
2	43605	1	1343	383	61	SMITHS CRINKLE CUT CHIPS CHICKEN 170G	
3	43329	2	2373	974	69	SMITHS CHIP THINLY S/CREAM&ONION 175G	
4	43330	2	2426	1038	108	KETTLE TORTILLA CHPSHNY&JLPNO CHILI 150G	

Customer Purchasing Behaviour Analysis

In [102…
```
# group by customer sales
```

In [101…
```
segment_sales = data.groupby(
    ['LIFESTAGE','PREMIUM_CUSTOMER']
)['TOT_SALES'].sum().sort_values(ascending=False)

segment_sales
```

Out[101…
```
LIFESTAGE               PREMIUM_CUSTOMER
OLDER FAMILIES          Budget              168363.25
YOUNG SINGLES/COUPLES   Mainstream          157621.60
RETIREES                Mainstream          155677.05
YOUNG FAMILIES          Budget              139345.85
OLDER SINGLES/COUPLES   Budget              136769.80
                        Mainstream          133393.80
                        Premium             132263.15
RETIREES                Budget              113147.80
OLDER FAMILIES          Mainstream          103445.55
RETIREES                Premium              97646.05
YOUNG FAMILIES          Mainstream           92788.75
MIDAGE SINGLES/COUPLES  Mainstream           90803.85
YOUNG FAMILIES          Premium              84025.50
OLDER FAMILIES          Premium              81958.40
YOUNG SINGLES/COUPLES   Budget               61141.60
MIDAGE SINGLES/COUPLES  Premium              58432.65
YOUNG SINGLES/COUPLES   Premium              41642.10
MIDAGE SINGLES/COUPLES  Budget               35514.80
NEW FAMILIES            Budget               21928.45
                        Mainstream           17013.90
                        Premium              11491.10
Name: TOT_SALES, dtype: float64
```

In [103…
```
#Average Spend per Customer
```

In [104…

```
avg_spend = data.groupby(
    ['LIFESTAGE','PREMIUM_CUSTOMER']
)['TOT_SALES'].mean().sort_values(ascending=False)

avg_spend
```

Out[104…

LIFESTAGE	PREMIUM_CUSTOMER	
MIDAGE SINGLES/COUPLES	Mainstream	7.647284
YOUNG SINGLES/COUPLES	Mainstream	7.558339
RETIREES	Premium	7.456174
OLDER SINGLES/COUPLES	Premium	7.449766
RETIREES	Budget	7.443445
OLDER SINGLES/COUPLES	Budget	7.430315
OLDER FAMILIES	Premium	7.322945
NEW FAMILIES	Mainstream	7.317806
	Budget	7.297321
YOUNG FAMILIES	Budget	7.287201
OLDER SINGLES/COUPLES	Mainstream	7.282116
OLDER FAMILIES	Budget	7.269570
YOUNG FAMILIES	Premium	7.266756
OLDER FAMILIES	Mainstream	7.262395
RETIREES	Mainstream	7.252262
NEW FAMILIES	Premium	7.231655
YOUNG FAMILIES	Mainstream	7.189025
MIDAGE SINGLES/COUPLES	Premium	7.112056
	Budget	7.074661
YOUNG SINGLES/COUPLES	Premium	6.629852
	Budget	6.615624

Name: TOT_SALES, dtype: float64

number of customer

Step 7: Customer Purchasing Behaviour Analysis

In [106…

```
customer_count = data.groupby(
    ['LIFESTAGE', 'PREMIUM_CUSTOMER']
)['LYLTY_CARD_NBR'].nunique().sort_values(ascending=False)

customer_count
```

Out[106…

```
LIFESTAGE               PREMIUM_CUSTOMER
YOUNG SINGLES/COUPLES   Mainstream          8088
RETIREES                Mainstream          6479
OLDER SINGLES/COUPLES   Mainstream          4930
                        Budget              4929
                        Premium             4750
OLDER FAMILIES          Budget              4675
RETIREES                Budget              4454
YOUNG FAMILIES          Budget              4017
RETIREES                Premium             3872
YOUNG SINGLES/COUPLES   Budget              3779
MIDAGE SINGLES/COUPLES  Mainstream          3340
OLDER FAMILIES          Mainstream          2831
YOUNG FAMILIES          Mainstream          2728
YOUNG SINGLES/COUPLES   Premium             2574
YOUNG FAMILIES          Premium             2433
MIDAGE SINGLES/COUPLES  Premium             2431
OLDER FAMILIES          Premium             2274
MIDAGE SINGLES/COUPLES  Budget              1504
NEW FAMILIES            Budget              1112
                        Mainstream           849
                        Premium              588
Name: LYLTY_CARD_NBR, dtype: int64
```

avg_packet per customer

In [108…

```
avg_packs = (
    data.groupby(['LIFESTAGE', 'PREMIUM_CUSTOMER'])['PROD_QTY'].sum()
    /
    data.groupby(['LIFESTAGE', 'PREMIUM_CUSTOMER'])['LYLTY_CARD_NBR'].nunique()
).sort_values(ascending=False)

avg_packs
```

Out[108…

```
LIFESTAGE               PREMIUM_CUSTOMER
OLDER FAMILIES          Mainstream          9.804309
                        Premium             9.749780
                        Budget              9.639572
YOUNG FAMILIES          Budget              9.238486
                        Premium             9.209207
                        Mainstream          9.180352
OLDER SINGLES/COUPLES   Premium             7.154947
                        Budget              7.145466
                        Mainstream          7.098783
MIDAGE SINGLES/COUPLES  Mainstream          6.796108
RETIREES                Budget              6.458015
                        Premium             6.426653
MIDAGE SINGLES/COUPLES  Premium             6.386672
                        Budget              6.313830
RETIREES                Mainstream          6.253743
NEW FAMILIES            Mainstream          5.087161
                        Premium             5.028912
                        Budget              5.009892
YOUNG SINGLES/COUPLES   Mainstream          4.776459
                        Budget              4.411485
                        Premium             4.402098
dtype: float64
```

In [ ]:

In [109…
```
avg_price = (
    data.groupby(['LIFESTAGE', 'PREMIUM_CUSTOMER'])['TOT_SALES'].sum()
    /
    data.groupby(['LIFESTAGE', 'PREMIUM_CUSTOMER'])['PROD_QTY'].sum()
).sort_values(ascending=False)

avg_price
```

Out[109…
```
LIFESTAGE               PREMIUM_CUSTOMER
YOUNG SINGLES/COUPLES   Mainstream          4.080079
MIDAGE SINGLES/COUPLES  Mainstream          4.000346
NEW FAMILIES            Mainstream          3.939315
                        Budget              3.936178
RETIREES                Budget              3.933660
                        Premium             3.924050
OLDER SINGLES/COUPLES   Premium             3.891695
NEW FAMILIES            Premium             3.886067
OLDER SINGLES/COUPLES   Budget              3.883299
RETIREES                Mainstream          3.842170
OLDER SINGLES/COUPLES   Mainstream          3.811578
MIDAGE SINGLES/COUPLES  Premium             3.763535
YOUNG FAMILIES          Budget              3.754840
                        Premium             3.750134
MIDAGE SINGLES/COUPLES  Budget              3.739975
OLDER FAMILIES          Budget              3.736009
                        Mainstream          3.726962
YOUNG FAMILIES          Mainstream          3.705029
OLDER FAMILIES          Premium             3.696649
YOUNG SINGLES/COUPLES   Premium             3.675060
                        Budget              3.667542
dtype: float64
```

In [124…
```
## Step 8: Data Visualisation
```

In [110…
```
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_style("whitegrid")
```

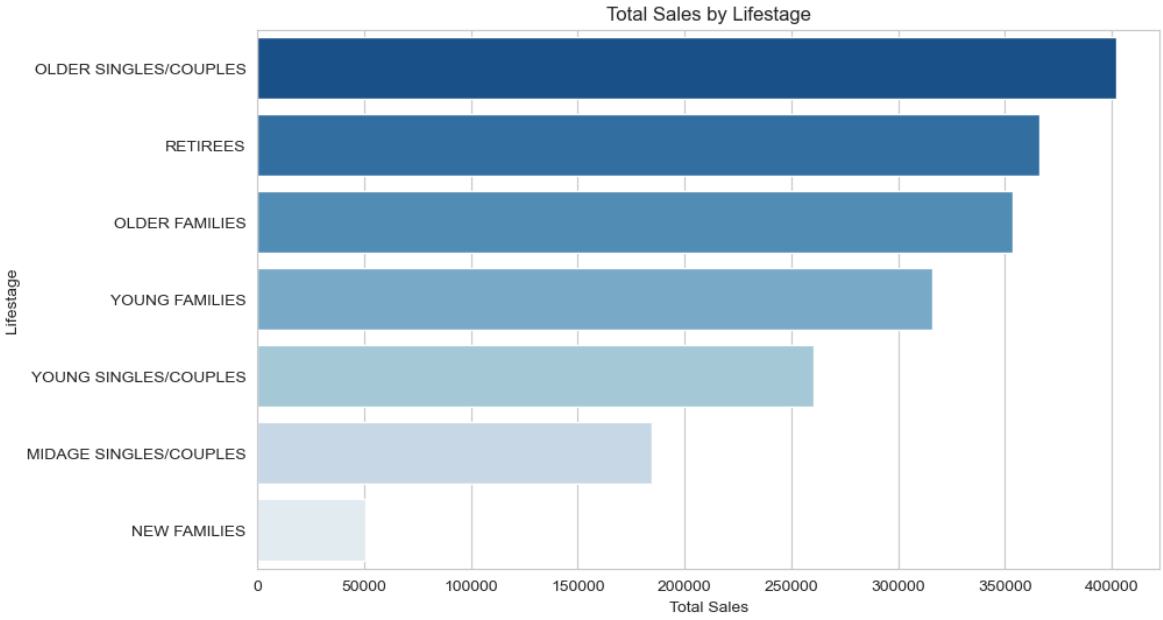
Key Insights

1. Older Families (Budget) generated the highest total sales.
2. Young Singles/Couples (Mainstream) represented the largest customer segment.
3. Older Families purchased the highest number of chip packs per customer.
4. Young Singles/Couples (Mainstream) paid the highest average price per pack, suggesting a preference for premium products.

In [111…
```
sales_by_lifestage = data.groupby('LIFESTAGE')['TOT_SALES'].sum().sort_values(as

plt.figure(figsize=(10,6))
```

```
sns.barplot(
    x=sales_by_lifestage.values,
    y=sales_by_lifestage.index,
    palette="Blues_r"
)

plt.title("Total Sales by Lifestage")
plt.xlabel("Total Sales")
plt.ylabel("Lifestage")

plt.show()
```

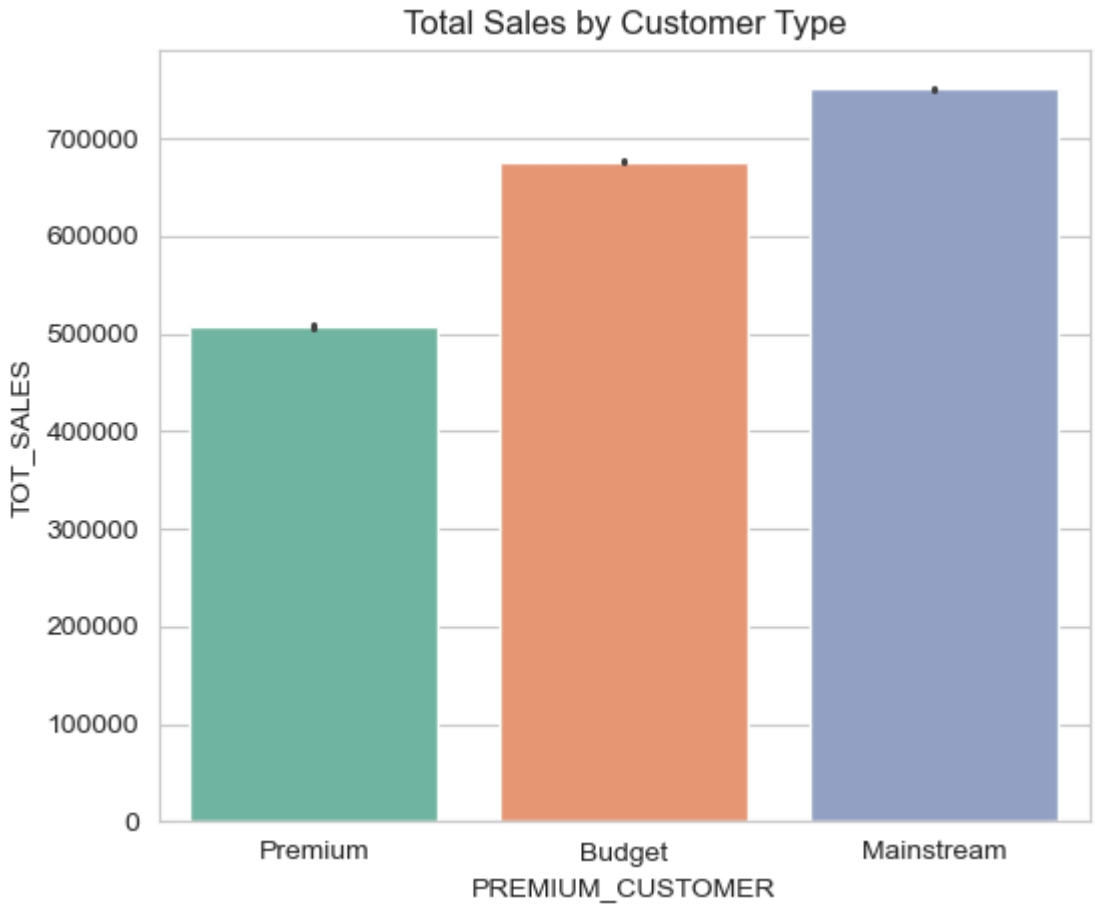
```
C:\Users\Balaji\AppData\Local\Temp\ipykernel_22324\1169693701.py:4: FutureWarnin
g:

Passing `palette` without assigning `hue` is deprecated and will be removed in v
0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effe
ct.

  sns.barplot(
```



In [113…

```
plt.figure(figsize=(6,5))

sns.barplot(
    data=data,
    x="PREMIUM_CUSTOMER",
    y="TOT_SALES",
    estimator=sum,
    palette="Set2"
)

plt.title("Total Sales by Customer Type")
plt.show()
```

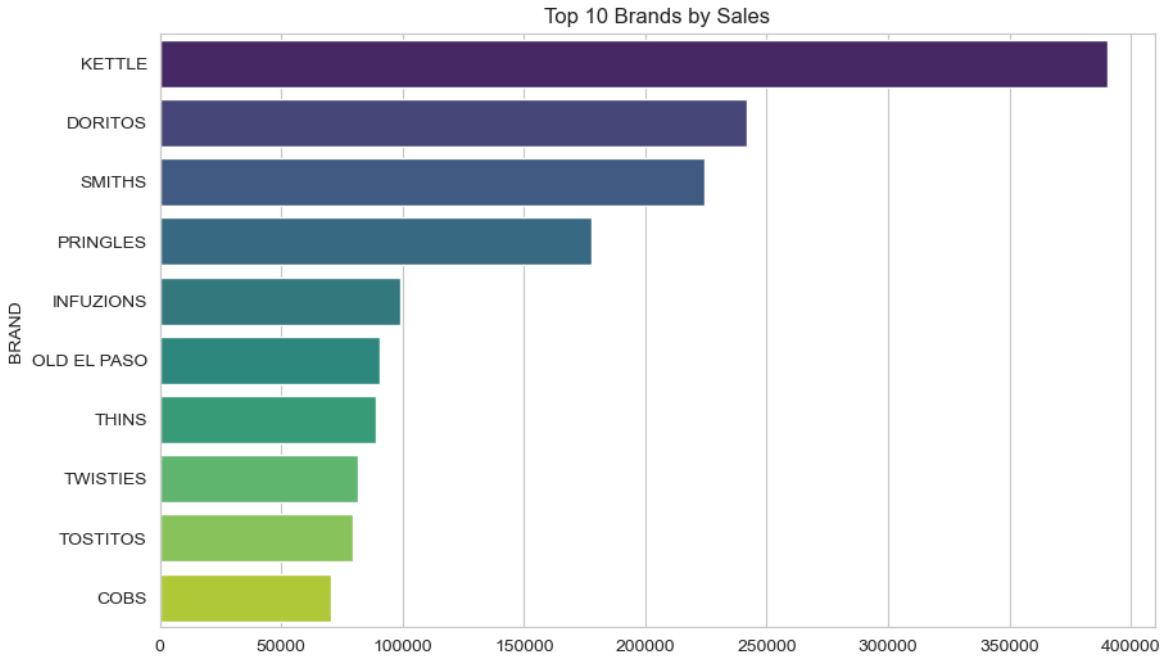
```
C:\Users\Balaji\AppData\Local\Temp\ipykernel_22324\2493052117.py:3: FutureWarnin
g:

Passing `palette` without assigning `hue` is deprecated and will be removed in v
0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effe
ct.

  sns.barplot(
```



In [114…

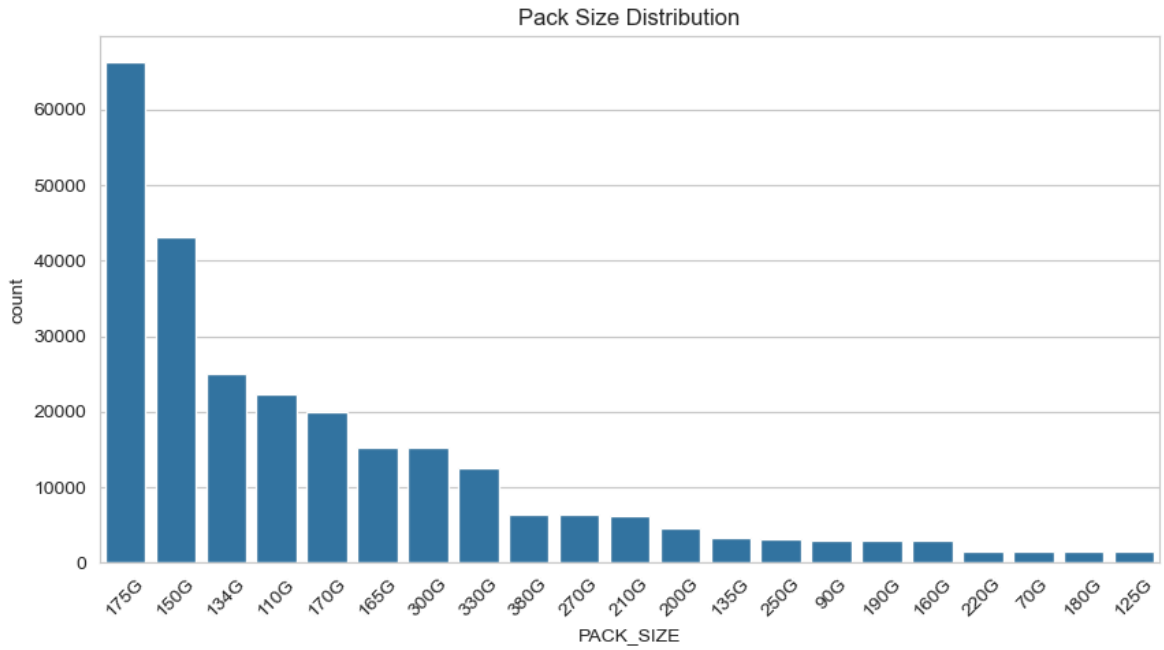
```
top_brands = data.groupby('BRAND')['TOT_SALES'].sum().sort_values(ascending=Fals

plt.figure(figsize=(10,6))

sns.barplot(
    x=top_brands.values,
    y=top_brands.index,
    palette="viridis"
)

plt.title("Top 10 Brands by Sales")

plt.show()
```

```
C:\Users\Balaji\AppData\Local\Temp\ipykernel_22324\4015682489.py:5: FutureWarnin
g:

Passing `palette` without assigning `hue` is deprecated and will be removed in v
0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effe
ct.

  sns.barplot(
```



In [115…

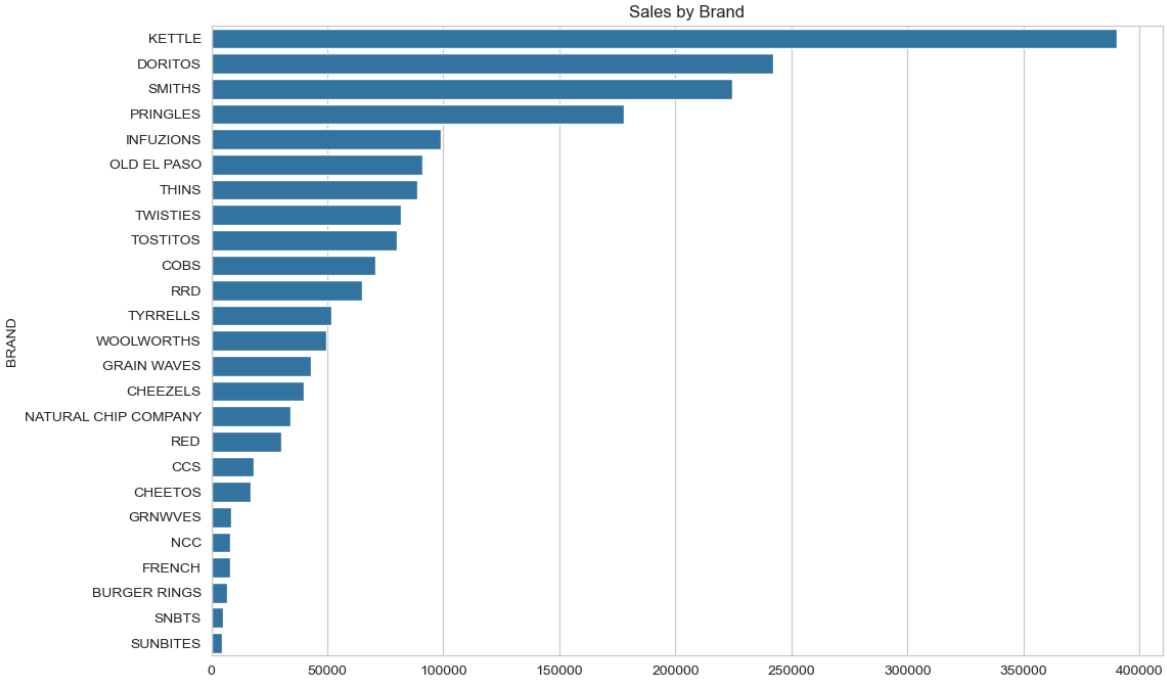
```
plt.figure(figsize=(10,5))

sns.countplot(
    data=data,
    x="PACK_SIZE",
    order=data["PACK_SIZE"].value_counts().index
)

plt.xticks(rotation=45)

plt.title("Pack Size Distribution")

plt.show()
```



In [116…

```
brand_sales = data.groupby("BRAND")["TOT_SALES"].sum().sort_values(ascending=Fal

plt.figure(figsize=(12,8))

sns.barplot(
```

```
    x=brand_sales.values,
    y=brand_sales.index
)

plt.title("Sales by Brand")

plt.show()
```



Business Recommendations

1. Target Older Families with family-size packs and value promotions.
2. Increase marketing of premium products to Young Singles/Couples (Mainstream).
3. Maintain strong product availability for Retirees.
4. Promote top-performing brands such as Kettle, Smiths, Doritos, and Pringles.

In [ ]: